

Diseño Digital Avanzado

**Unidad 6 - Diseños Basados en Maquinas
de Estado**

Dr. Ariel L. Pola
ariel.pola@mi.unc.edu.ar
December 10, 2024

Tabla de Contenidos

1. Contenidos Temáticos
2. Diseño con Máquinas de Estado
3. Ejemplos de Arq. Mult. en Tiempo
4. Secuenciamiento y Control
5. Rep. Algorítmica de máquinas de estado
6. Optimización de FSM
7. Diseño para Verificación
8. Métodos para Reducción de Potencia



The image shows a detailed 3D rendering of an Intel Agilex FPGA chip. The chip is a square package with various components labeled on its surface. At the top, it says 'Intel® Agilex™ FPGA'. Below this, there's a large black square labeled '10nm FPGA Fabric'. To the left of the fabric is a blue chip labeled 'CPU Core' and a green chip labeled 'PCIe Gen 5'. To the right is a red chip labeled 'Other Chiplet'. Below the fabric is a silver chip labeled 'DDR'. Above the fabric is a black chip labeled 'HBM'. There are also several yellow chips labeled 'EMIB' (Embedded Multi-Die Interconnect Bridge) connecting different parts of the chip. The background is a dark blue network of lines and nodes, suggesting a digital or interconnected environment.

Contenidos Temáticos

Unidad 6 Diseño de Máquinas de Estado

- Ejemplos de arquitecturas multiplexadas en tiempo.
- Arquitecturas seriales a nivel de bit y palabra.
- Arquitecturas secuenciales.
- Máquinas de estado finitas.
- Codificación de los estados.
- Máquinas de Moore y Mealy.
- Guías para la codificación de las máquinas de estado.
- Distintas representaciones gráficas de las mismas.
- Ejemplos de máquinas de estado.
- Diseño para verificación.
- Metodología.
- Métricas de cobertura.
- Métodos de reducción de potencia.

[illegible]

Diseño de Máquinas de Estado

Introducción

- En las aplicaciones que son mapeadas en arquitecturas dedicadas paralelas, no es necesario utilizar el Hardware en cada ciclo de clock.
- Estos diseños utilizan los recursos de Hardware mínimos requeridos y luego comparten estos recursos en múltiples procesamiento del algoritmo en diferentes ciclos de clock.
- Un diseño digital síncrono que comparte bloques de Hardware para procesamiento en diferentes ciclos de clock, requiere un **controlador**. Este controlador implementa un scheduler o planificador que dirige el uso de los recursos multiplexando en el tiempo.
- Existen diferentes opciones para el controlador, entre ellos se encuentran las máquinas de estado de Moore y de Mealy que se tratan en este curso.
- En la máquina **Mealy**, la salida y el siguiente estado son funciones de la entrada y del estado actual.
- En la máquina **Moore**, el estado actual y la entrada solo computan el siguiente estado y la salida solo depende del estado actual.

Diseño de Máquinas de Estado

Introducción

- Las máquinas Moore proporcionan una entrada de control estable a la ruta de datos para un **ciclo de reloj completo**.
- En diseños que usan la máquina Mealy, la salida puede cambiar con el cambio de entrada y puede **no permanecer estable durante un ciclo completo**.
- En el diseño digital de procesamiento de señal, estas señales de salida se utilizan para seleccionar lógica en un datapath. Estas señales entonces son críticas respecto del tiempo y deberían ser estables durante el ciclo completo. Las señales de salida estables pueden obtenerse registrando la salida de una máquina de Mealy.
- Existen diferentes métodos para la codificación de los estados, estos métodos afectan al tamaño de los registros de estados. Una máquina de estado one-hot utiliza un flip-flop por cada estado. Esta opción es atractiva, ya que resulta en un análisis de tiempo simple y agregar o eliminar estados nuevos es trivial. Esta máquina tiene especial interés en FPGA ya que disponen de gran cantidad de flip-flops.
- Si el objetivo es conservar el número de flip-flops del registro de estados, se puede utilizar una máquina codificada-binaria.

Ejemplos de Arq. Mult. en Tiempo



Arquitecturas Multiplexadas en el tiempo

Arquitecturas Bit-Serial y Digit-Serial

Arquitecturas Bit-Serial

- Las aplicaciones que requieren mapeo en el tiempo con recursos de hardware compartidos, son representadas por simples gráficos de flujo de datos (DFGs) la lógica respectiva de planificación de estados.
- La arquitectura Bit-Serial trabaja a una tasa bit-a-bit. Esto resulta particularmente de interés cuando el dato es la entrada a un sistema en una tasa bit-a-bit en una interfaz serie. La interfaz brinda al diseñador la motivación para la arquitectura bit-serial.
- El procesamiento de bit-a-bit de datos recibidos de manera serial, minimiza el área y muchas veces también reduce la complejidad del diseño ya que en este caso no es necesario mantener el ordenamiento de bits en formato de palabras para su procesamiento.

Arquitecturas Multiplexadas en el tiempo

Arquitecturas Bit-Serial y Digit-Serial

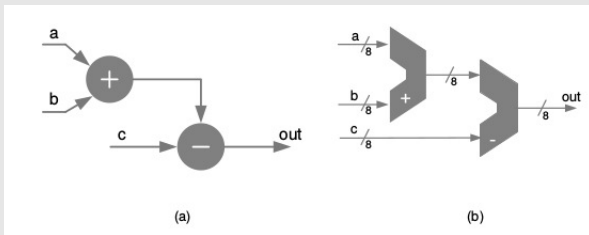
Arquitecturas Digit-Serial

- Una extensión de la arquitectura bit-serial es la digit-serial donde la arquitectura divide un operando de N -bits en $P = N/M$ – dígitos-bit que alimentan al sistema de forma serial, y el datapath completo es de ancho P , donde P es un entero.
- La elección de P depende del requerimiento de la tasa de muestreo (throughput) de arquitectura y puede ser de ancho 1 a N bits.
- Es pertinente destacar que, como consecuencia en la densidad de los dispositivos, el área no suele ser un requerimiento muy importante, por lo que el diseñador no debería involucrarse en las complicaciones que implica el diseño bit-serial. Solo los diseños que de forma natural encajan en un procesamiento bit-serial deberían ser mapeados a estas arquitecturas.

Arquitecturas Multiplexadas en el tiempo

Arquitecturas Bit-Serial y Digit-Serial

Ejemplo de Arquitectura Multiplexada en el tiempo



Se muestra un diseño de FDA en el cual se asume que la frecuencia de muestreo es la misma que la del circuito. El diseño se implementa con pipelines para incrementar el desempeño de throughput de la arquitectura. La técnica sugiere el uso de CSA y árboles de compresión para minimizar el uso de CPA.

Arquitecturas Multiplexadas en el tiempo

Arquitecturas Bit-Serial y Digit-Serial

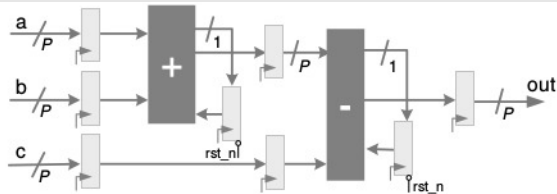
Arquitecturas Digit-Serial

- Ahora asumimos que en el ejemplo anterior, la frecuencia de muestreo es ocho veces más lenta que la frecuencia del circuito. Esta relación implica que el hardware se puede diseñar para compartir recursos 8 veces.
- Cuando se mapea el DFG para $f_c = 8f_s$ el diseño se optimiza y se transforma en una arquitectura bit-serial en la cual las operaciones son efectuadas en una tasa bit-a-bit, utilizando de forma efectiva 8 clocks para cada muestra de entrada.
- En la siguiente filmína se muestra la implementación de Hardware del algoritmo, en la cual se utiliza un diseño bit-serial $P = 1$. El diseño consiste en un FA y un FS (Full Subtractor) reemplazando el sumador y substracción del diseño FDA.
- Asociadas a los FA y FS para adición de 1 bit encontramos dos Flip-Flops que se conectan al carry y borrow anteriores nuevamente al FA y al FS para las operaciones con las siguientes muestras.
- Cada 8 clocks se vacían los flip-flops cuando el nuevo dato de 8 bits es ingresado al circuito. La suma del FA se pasa al FS a través del pipeline.
- El datapath de un bit no requiere el empaquetado de bits intermedios para ninguna operación.

Arquitecturas Multiplexadas en el tiempo

Arquitecturas Bit-Serial y Digit-Serial

Ejemplo de Arquitectura Multiplexada en el tiempo



Es interesante notar que la arquitectura se puede pasar a digit-serial alimentando dígitos en lugar de bits al diseño. Los FF para carry y borrow quedarían iguales, mientras que los registros del sumador, restador y otros pipelines pueden cambiar al tamaño del dígito. Por ejemplo si el dígito se considera como de 4 bits ($P = 4$), la arquitectura tomará dos ciclos en procesar dos grupos de entradas de 4-bits produciendo la salida [3:0] en el primer ciclo y luego [7:4] en el segundo.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Arquitectura Secuencial

- En muchos diseños el algoritmo es tan regular que el hardware puede ser diseñado de manera secuencial para efectuar cada iteración en hardware compartido.
- El hardware ejecuta el algoritmo como si se tratara de un programa de software, secuencialmente implementando una iteración del algoritmo en un ciclo de clock. Esta arquitectura resulta en una optimización de hardware y normalmente requiere una lógica de control simple.
- Esos algoritmos que pueden ser mapeados a arquitecturas secuenciales pueden transformarse a sistemas paralelos. Las arquitecturas sistólicas pueden ser diseñadas para mapeo optimizado de Hardware.

Arquitecturas Multiplexadas en el tiempo

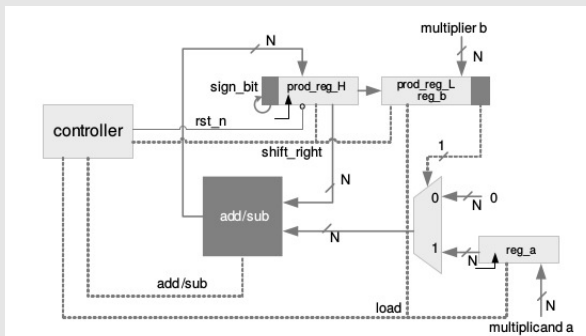
Arquitectura Secuencial

- El próximo ejemplo considera el diseño de un multiplicador secuencial. El mismo multiplica dos números signados de N -bits a y b en N ciclos de clock. En el primer $N - 1$ ciclo el multiplicador genera el producto parcial i (PP i) para un bit $axb(i)$ y suma su versión desplazada a una suma.
- Para la multiplicación de signo por signo, el bit más significativo del multiplicador b tiene peso negativo. Entonces el valor desplazado $N - 1$ del PP final para $axb(N - 1)$ se resta de la suma. Luego de N ciclos de clock la suma almacena el producto final de la operación.
- El algoritmo en pseudo-código sería:
 $sum = 0$
 $for(i = 0; i < N; i++)$
 $sum+ = axb[i]x2^i$
 $sum- = axb[N - 1]x2^{N-1}$
 $prod = sum$

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Ejemplo de Arquitectura Secuencial: Multiplicador



La figura muestra la implementación del multiplicador $N \times N$ secuencial. El shift register de $2N$ -bit $prod_reg$ se puede ver como dos registros concatenados: el registro con los N LBSs es $prod_reg_L$ y el registro con los N MSBs es $prod_reg_H$.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Ejemplo de Arquitectura Secuencial: Multiplicador

- Un sumador/restador de N-bits suma los PPs intermedios y resta el PP final de una suma continua que se guarda en *prod_reg_L*.
- El PP *i* se ejecuta de manera sucesiva a través del desplazamiento de la suma en cada clock. .El paso a paso sería:
 - P1 : Cargar los registros con los valores apropiados :*reg_a* se carga con el multiplicando *a*, *prod_reg_L* (*reg_b*) con el multiplicador *b* y *prod_reg_H* se configura en 0.
 - P2 : Cada ciclo suma el multiplicando al acumulador si el bit *i* del multiplicador es 1 y 0 si es 0. En cada ciclo, *prod_reg* se desplaza en 1. Para la multiplicación de signos, se utiliza un desplazamiento hacia la derecha escribiendo nuevamente el MSB de *prod_reg* a su posición mientras todos los bits se desplazan a la derecha. Para multiplicación sin signo se realiza un desplazamiento lógico de uno en cada paso.
 - P3 : Para la multiplicación signada, en el ciclo N^{th} , el sumador/substractor se configura para resta, mientras que para multiplicación no signada se ejecuta la suma en este ciclo también.
 - P4 : Luego de N ciclos *prod_reg* guarda el resultado final del producto.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Optimización del multiplicador secuencial

- La arquitectura anteriormente propuesta puede ser optimizada moviendo el CPA fuera del lazo de realimentación. Los resultados de la acumulación son almacenados en la suma parcial y carry formados en *prod_reg_HS* y *prod_reg_HC* respectivamente.
- El árbol de compresión toma los valores almacenados en estos registros junto con el nuevo PP *i* y los comprime en dos capas. Esta operación siempre produce un bit de producto libre (como se muestra en la figura siguiente b).
- La suma parcial y carry que se derivan de la compresión se almacenan una posición hacia la derecha. En *N* ciclos de clock el diseño produce *N* LSB del producto, mientras que *N* MSBs en formato de suma y carry se mueven a un conjunto de registros que trabajan con un clock más lento (el de muestreo f_s) y luego son sumados empleando un CPA lento.
- El árbol de compresión tiene únicamente un delay del Full Adder y trabaja a una tasa más rápida, permitiendo que el diseño se adapte a aplicaciones de alto desempeño.

Arquitectura Secuencial

(a)

(b)

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Ejemplo con Filtro FIR

- En este ejemplo se explica la implementación de un filtro FIR de 5 coeficientes que implementa la siguiente ecuación en diferencias:

$$y_n = h_0x_n + h_1x_{n-1} + h_2x_{n-2} + h_3x_{n-3} + h_4x_{n-4}$$

- Para la implementación secuencial, asumimos que f_c es cinco veces más rápida que la frecuencia de muestreo f_s . Esto implica que luego de muestrear un valor de entrada a f_s , el diseño corriendo a f_c tiene 5 ciclos de clock para procesar una muestra de salida.
- Una implementación FDA requeriría 5 multiplicadores y 4 sumadores. Utilizando esta arquitectura, el diseño operando a f_c procesa la salida en un ciclo y luego esperaría durante cuatro ciclos la siguiente muestra de entrada. Esto es un claro desperdicio de recursos, por lo que se prefiere una arquitectura multiplexada en el tiempo.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Ejemplo con Filtro FIR

- Una implementación con arquitectura secuencial implementa el mismo diseño utilizando un bloque MAC (Multiplier and Accumulator). El pseudo-código sería:

$acc = 0$

$for(i = 0; i < 4; i++)$

$acc += h_i x_{n-i}$

$y_n = acc$

- El diseño se puede optimizar moviendo el CPA fuera de las operaciones de multiplicación y acumulación.
- El resultado del árbol de compresión se almacenan de manera parcial en formato de suma y carry en dos grupos de registros de acumulación acc_s y acc_c y también son realimentados al árbol de compresión para la operación MAC.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

Ejemplo con Filtro FIR

- Los resultados parciales se mueven a dos registros que trabajan a una tasa menor f_s . El CPA suma los dos valores al clock más lento f_s . El pseudo-código para el diseño optimizado es:

$acc_s = 0$

$acc_c = 0$

$for(i = 0; i < 4; i++)$

$\{acc_s, acc_c\} + = h_i x_{n-i}$

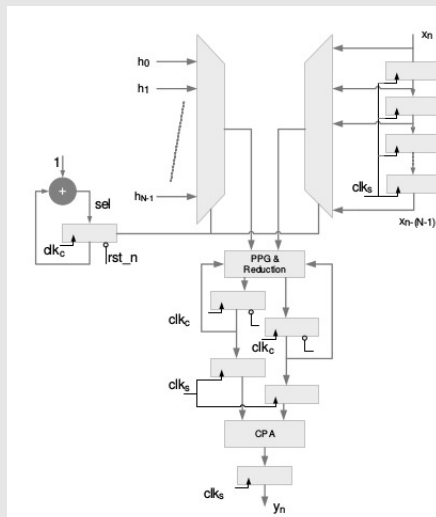
$y_n = acc_s + acc_c$

- En la siguiente filmína se muestra el diagrama del diseño del FIR para N-coeficientes, en este diseño se asume que $f_c/f_s = N$.
- El diseño tiene una línea de delays de entrada que muestrea la nueva entrada en cada clock f_s y desplaza todos los valores por un registro.
- Los coeficientes pueden ser almacenados en una ROM o utilizar registros si el diseño soporta coeficientes adaptativos.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

FIR Secuencial



Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

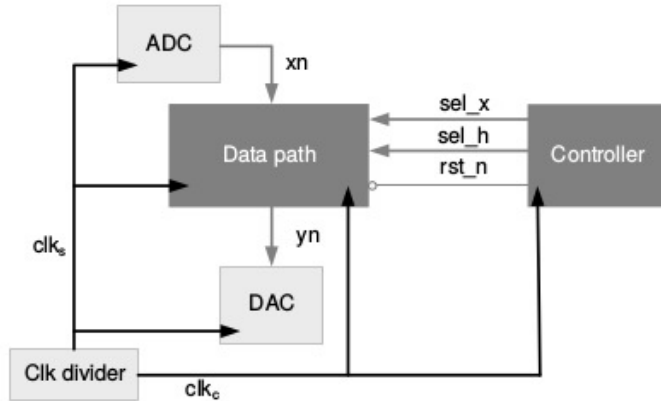
Ejemplo con Filtro FIR

- El diseño de hardware asume que el clock de muestreo se deriva de manera síncrona del clock del circuito. El shift register que almacena las muestras de entradas anteriores y actuales utilizan el clock de muestreo al igual que el registro final de salida.
- En cada flanco positivo del clock de muestreo, cada registro almacena un nuevo valor previamente almacenado en el registro sobre el, con el registro superior almacenando la nueva muestra de entrada.
- El datapath ejecuta el procesamiento en el circuito con el clock del circuito clk_C que es N veces más rápido que el clock de muestreo. El diseño de top-level consiste en el datapath y el controlador que se muestra en la siguiente figura.
- El controlador vacía los registros de la suma parcial y el carry y comienza a generar las señales de selección sel_h y sel_x para los dos multiplexores, comenzando de 0 e incrementando de a 1 con cada flanco positivo del clock clk_C .
- En el ciclo N^{th} se genera la salida final y el controlador reinicia el proceso con una nueva muestra de entrada.

Arquitecturas Multiplexadas en el tiempo

Arquitectura Secuencial

FIR Secuencial - Datapath y Controlador



Secuenciamiento y Control



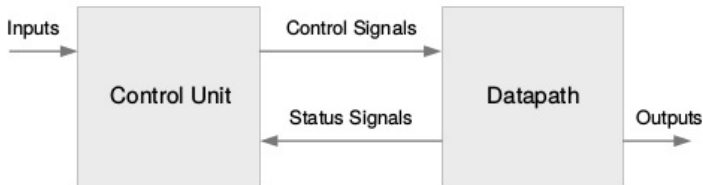
Secuenciamiento y Control

Introducción

- En general, las arquitecturas multiplexadas en el tiempo consisten en datapath y unidad de control. El datapath es la etapa del procesamiento computacional y consiste en registros, multiplexores, de-multiplexores, ALUs, multiplicadores, shifters, circuitos combinacionales y buses. Estos recursos de Hardware son compartidos a lo largo de los diferentes procesos del algoritmo.
- El controlador genera las señales de control para la selección de estos operandos en una secuencia predefinida. La secuencia está determinada por el diagrama de flujo de datos o flujo del algoritmo.
- Algunas de las operaciones en la secuencia pueden depender de resultados de procesos anteriores, con lo cual las señales de estado son realimentadas a la unidad de control.
- La secuencia de las operaciones puede depender también de señales de entrada de otros módulos de un sistema.

Secuenciamiento y Control

Diagrama básico de arquitectura multiplexada en el tiempo



La figura muestra los dos bloques básicos. La unidad de control implementa la planificación de las operaciones utilizando una máquina de estado finita

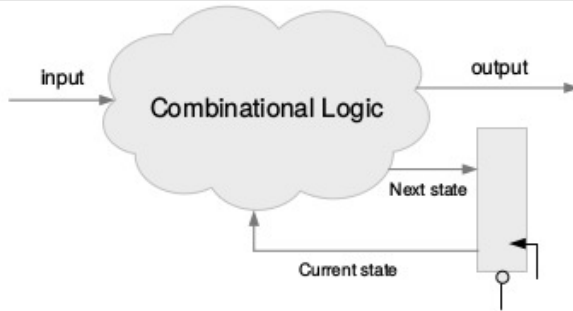
Introducción

- Un diseño digital síncrono normalmente ejecuta una secuencia de operaciones en datos almacenados en registros y memorias de su datapath bajo la dirección de una máquina de estados finita (FSM).
- La FSM es un circuito digital con un registro de N-estados para almacenar el valor actual de la secuencia de operaciones. Un registro de estados de N-bit puede tener 2^N posibles valores, con lo cual el número de estados posibles en la secuencia es finito.
- La FSM puede describirse utilizando un diagrama de burbujas (estado) o un cuadro de máquina de estado algorítmica (ASM).
- La implementación de hardware de una FSM tiene dos componentes, una lógica combinacional y una lógica secuencial. La lógica combinacional procesa la salida y el siguiente estado basado en el estado actual y la entrada, mientras que la parte secuencial tiene el registro de estado con posibilidad de reset.

Secuenciamiento y Control

Máquinas de Estado Finitas

Diagrama Básico de una FSM



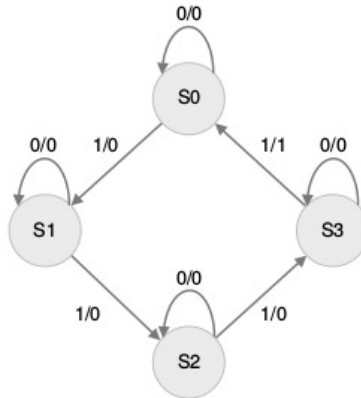
Ejemplo FSM

- El ejemplo que se trabaja a continuación es el diseño de una FSM que cuenta cuatro 1s en una interfaz serie y genera un 1 a la salida. Se recibe un bit en una interfaz serie en cada ciclo de clock.
- Se tienen cuatro estados en total S0...S3. S0 representa el estado inicial en el cual el número de 1s recibidos en la interfaz es cero. Un 0/0 en la curva de transición indica que, en el estado S0, si la entrada de la interfaz es 0 entonces la máquina de estado se mantiene en ese estado y genera un 0 en la salida.
- En el mismo estado inicial S0, si se recibe un 1 en la entrada la FSM cambia su estado a S1 y genera un 0 en la salida. Esta transición y relación entrada-salida se representa como 1/0 en la línea o curva con una flecha indicando la transición de estado de S0 a S1. No existe un nuevo cambio de estado si se continúa recibiendo ceros, mientras que otro 1 en la entrada provoca la transición a S2.
- La máquina se mantiene en S2 si se reciben ceros mientras que otro uno causa la transición a S3. En este estado, si la FSM recibe un 1 (el 4to 1 recibido), genera un 1 en la salida y regresa al estado inicial S0 donde comienza a contar nuevamente.

Secuenciamiento y Control

Máquinas de Estado Finitas

Ejemplo FSM - Diagrama de Estados



One-hot vs Asignación Binaria

- Existen varias formas de codificar los estados en un FSM. La codificación define una secuencia única de números para representar cada estado. Por ejemplo en una máquina “one-hot” con N estados existen N bits para representar estos estados. Un 1 aislado en una ubicación de bit única representa un estado particular de la máquina. Esto requiere un registro de estados de N bits.
- Si bien la máquina de estados one-hot resulta en la implementación de lógica simple para las transiciones de estado, requiere N flip-flops. La alternativa de asignación binaria requiere $\log_2(N)$. Esto requiere menor cantidad de flip-flops pero la lógica que decodifica los estados y genera los estados siguientes es más compleja en este tipo de codificación.
- La máquina de estados con codificación one-hot es más útil en el caso de las FPGA que tienen una gran cantidad de flip-flops. La técnica también evita fan-out importante que se puede derivar de la codificación binaria de registros de estados, que no se sintetiza bien en FPGAs.

One-hot vs Asignación Binaria

- Una variante llamada 'casi one-hot' es similar aunque con la excepción de que el estado inicial es codificado como todos en cero. Esto ayuda a resetear el registro de estados. En otra variante se utilizan dos bits en lugar de uno.
- Para los diseños de baja potencia, el objetivo es reducir la distancia hamming entre transiciones de estado. Esto requiere que el diseñador conozca de manera estadística el patrón más probable de transición de estado. El patrón se codifica utilizando código gray. Estos cuatro tipos de codificación se muestran en la tabla.
- En el ejemplo anterior de FSM, si los estados se codificaran con one-hot, se necesitaría un registro de estados de tamaño 4 donde cada flip-flop del registro tendría asignado un estado de la FSM.

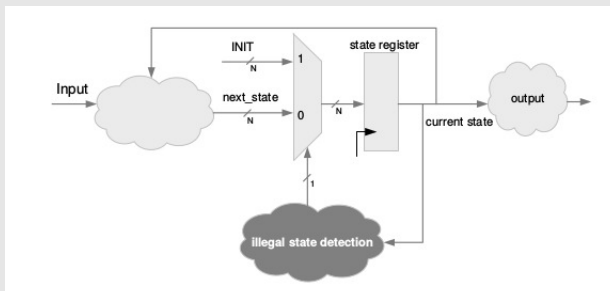
Binary	One-hot	Almost one-hot	Gray
Parameter [1:0]	Parameter [3:0]	Parameter [2:0]	Parameter [1:0]
S0 = 2'd00	S0 = 4'b0001	S0 = 3'b000	S0 = 2'b00
S1 = 2'd01	S1 = 4'b0010	S1 = 3'b001	S1 = 2'b01
S2 = 2'd10	S2 = 4'b0100	S2 = 3'b010	S2 = 2'b11
S3 = 2'd11	S3 = 4'b1000	S3 = 3'b100	S3 = 2'b10

Secuenciamiento y Control

Codificación de Estados

Como consecuencia de la codificación one-hot, solo un bit del registro de estados puede ser 1. El diseñador debe gestionar los estados ilegales verificando si más de un bit en el registro de estados es 1. En diseños críticos se utiliza lógica independiente para verificar cuando la FSM se encuentra en un estado ilegal y entonces la lleva al estado de reset.

Ejemplo FSM - One-Hot



Secuenciamiento y Control

Máquinas de Estado de Moore y de Mealy

Mealy FSM

- En una implementación de máquina de Mealy, la salida de la FSM es una función tanto del estado actual como de la entrada. Esto se prefiere en aplicaciones donde todas las entradas a una FSM son salidas registradas desde otros bloques y la generación del siguiente estado no es muy compleja. Resulta en una lógica combinacional simple con caminos críticos cortos.
- En los casos en los que la entrada a la FSM es asíncrona (es decir, que puede cambiar dentro de un ciclo de clock), las salidas de la máquina de estados también será asíncrona. Esto no es deseable especialmente en diseños en los cuales la salida de la FSM son señales de control para el datapath.
- Las unidades de procesamiento del datapath también son circuitos combinacionales, de manera que una señal de control asíncrona implica que el control puede no ser estable durante el ciclo de clock completo. Esto no es deseable ya que el datapath espera que todas las señales de control sean válidas por un ciclo completo de clock.

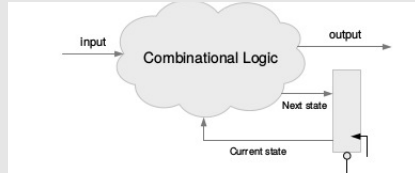
Secuenciamiento y Control

Máquinas de Estado de Moore y de Mealy

Mealy FSM

- En los diseños en los cuales la estabilidad de la señal de salida no es un requerimiento, se prefiere la máquina de Mealy ya que resulta en menor cantidad de estados.

Mealy FSM



La nube combinacional procesa el siguiente estado y la salida está basada en el estado actual y la entrada. El registro de estado tiene un reset asíncrono para inicializar la FSM en cualquier momento que sea necesario.

Secuenciamiento y Control

Máquinas de Estado de Moore y de Mealy

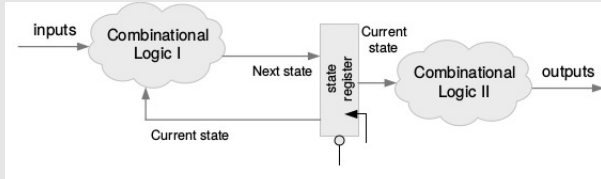
Moore FSM

- En una implementación de una máquina de Moore, las salidas son únicamente función del estado actual. Debido a que el estado actual está registrado, las señales de salidas se encuentran estables durante un ciclo completo de clock. En contraste con la máquina de Mealy, la salida estará retrasada por un ciclo de clock.
- La elección entre la implementación de una FSM de Moore o Mealy es del diseñador y es independiente del problema de diseño. En muchos casos es una decisión directa la de elegir entre las dos opciones. En instancias en las que alguna de las entradas puede tener un glitch y las salidas deben ser estables por un ciclo completo de clock, el diseñador debería siempre utilizar una máquina de Moore.
- En la siguiente filmina se muestra un ejemplo de una máquina de Moore. La lógica combinacional logic-1 de la FSM computa el siguiente estado basada en las entradas y el estado actual y la etapa combinacional logic-2 de la FSM procesa la salida basándose en el estado actual.

Secuenciamiento y Control

Máquinas de Estado de Moore y de Mealy

Moore FSM



Secuenciamiento y Control

Guía para la codificación de FSM

Separar el diseño en Datapath y control

- El diseño completo de hardware normalmente consiste de datapath y controlador. El controlador se implementa como una máquina de estado. Desde el punto de vista de la síntesis, las partes de datapath y control tienen distintos objetos de diseño.
- El datapath se sintetiza para lograr objetivos de cierre de timing mientras que el controlador se sintetiza para ocupar la menor cantidad de área. El diseñador debería mantener la parte de la lógica de la FSM y el datapath en distintos módulos y luego sintetizarlos por separado para poder establecer los objetivos del diseño.

Codificación de FSM en bloques de procedimiento

- La lógica en un módulo de FSM se codifica utilizando uno o dos bloques always. Se prefieren dos bloques always, uno en el cual se implemente la lógica secuencial que asigna el siguiente estado al registro de estado y otro en el cual se implemente la lógica combinacional que procesa el estado siguiente. Si la salida es de fácil procesamiento se puede computar de manera separada como asignación continua.

Secuenciamiento y Control

Guía para la codificación de FSM

Codificación de estados

- Cada estado en una FSM tiene un código asignado. Desde la perspectiva de la legibilidad, el diseñador debería utilizar etiquetas utilizando parámetros o directivas *define* para brindar información sobre los estados posibles. Se prefiere el uso de parámetros ya que el alcance del parámetro está limitado al módulo en el cual se define mientras que *define* es de alcance global.
- El uso de parámetros permite al diseñador utilizar los mismos nombres de estado con diferente codificación en otros módulos.
- El diseñador deberá elegir la mejor opción de codificación entre one-hot, casi one-hot, gray o binario. También deberá permitir a la herramienta de síntesis codificar los estados seleccionando las directivas apropiadas. Algunas herramientas permiten elegir el método de codificación especificando en la declaración de parameter.
- El datapath se sintetiza para lograr objetivos de cierre de timing mientras que el controlador se sintetiza para ocupar la menor cantidad de área. El diseñador debería mantener la parte de la lógica de la FSM y el datapath en distintos módulos y luego sintetizarlos por separado para poder establecer los objetivos del diseño.

Secuenciamiento y Control

Guía para la codificación de FSM

Directivas de Síntesis

- En muchos casos en donde se utilizan algunos estados de todos los posibles en el registro de estados, el diseñador puede indicar a la herramienta de síntesis que ignore los estados utilizados mientras se optimiza la lógica. Por ejemplo, para Synopsys se utiliza la directiva:

```
1 case(state) // Synopsys full_case
```

- Agregar esa directiva a la instrucción case indica a la síntesis que debe tratar todos los casos que no están definidos de manera explícita como 'don't care' para optimización. El diseñador sabe que los casos no definidos no van a suceder nunca. Considere el siguiente ejemplo:

```
1 always @(*) begin
2   case (cntr) // Synopsys full_case
3     2'b00: out = in1;
4     2'b01: out = in2;
5     2'b10: out = in3;
6   endcase
```

- El usuario controla a la herramienta de síntesis indicando que la señal cntr nunca va a tomar el valor no utilizado 2'b11. La herramienta optimiza la lógica considerando este caso como 'don't care'.

Ejemplo de Directivas de Síntesis

- De manera similar la sentencia Synopsys `parallel_case` se utiliza cuando todos los casos en un `case`, `casez` o `casex` son mutuamente excluyentes y el diseñador desea que sean evaluados de manera paralela o el orden en el cual serán evaluados no es importante. La directiva también indica que todos los casos deben ser evaluados individualmente y de manera paralela:

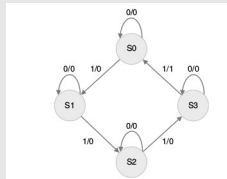
```
1 always @(*) begin
2     // Code for setting the output to default comes here
3     casez (intr_req) // Synopsys parallel_case
4         3'b??1:
5             begin // Check bit 0 while ignoring rest
6                 // Code for interrupt 0 comes here
7             end
8         3'b?1?:
9             begin // Check bit 1 while ignoring rest
10                // Code for interrupt 1 comes here
11            end
12        3'b1??:
13            begin // Check bit 2 while ignoring rest
14                // Code for interrupt 2 comes here
15            end
16    endcase // casez (intr_req)
17 end
```

Secuenciamiento y Control

Guía para la codificación de FSM

Directivas de Síntesis

- Mientras se utilice one-hot o casi one-hot como método de codificación, la directiva de Synopsis `full_case_parallel_case` significa que todos los casos no se solapan y solo un bit del registro de estados va a estar seteado y la herramienta debería considerar el resto de los patrones de bits del registro de estados como 'don't care'.
- Esta directiva genera la lógica más óptima para la FSM. Se prefiere el uso de la directiva `parallel_case` y `full_case` antes que el default en el case. La directiva de default debería utilizarse solo en la simulación y luego deshabilitada para la síntesis utilizando la directiva de compilador.
- El próximo ejemplo implementa la FSM de la figura utilizando codificación binaria.



Secuenciamiento y Control

Guía para la codificación de FSM

Ejemplo de Aplicación de la Guía para FSM

```
1 // This module implements FSM for the detection of
2 // four ones in a serial input stream of data
3 module fsm_mealy(
4     input    clk, //system clock
5     input    reset, //system reset
6     input    data_in, //1-bit input stream
7     //1-bit output to indicate 4 ones are detected or not
8     output reg four_ones_det
9 );
10 // Internal Variables
11 reg [1:0]    current_state; //4-bit current state
12     register
13 reg [1:0]    next_state; //4-bit next state register
14 // State tags assigned using binary encoding
15 parameter STATE_0 =2'b00;
16 parameter STATE_1 =2'b01;
17 parameter STATE_2 =2'b10;
18 parameter STATE_3 =2'b11;
```

```
1 // Next State Assignment Block
2 // This block implements the combination cloud of next
3 // state assignment logic
4 always @(*)
5     begin : next_state_bl
6     case(current_state)
7         STATE_0 : begin
8             if(data_in) begin
9                 //transition to next state
10                 next_state = STATE_1;
11                 four_ones_det = 1'b0;
12             end
13         else begin
14             //retain same state
15             next_state = STATE_0;
16             four_ones_det = 1'b0;
17         end
18     end // case: STATE_0
```

Secuenciamiento y Control

Guía para la codificación de FSM

Ejemplo de Aplicación de la Guía para FSM

```
1  STATE_1: begin
2      if (data_in) begin
3          //transition to next state
4          next_state = STATE_2;
5          four_ones_det = 1'b0;
6      end
7      else begin
8          //retain same state
9          next_state = STATE_1;
10         four_ones_det = 1'b0;
11     end
12 end // case: STATE_1
13 STATE_2 : begin
14     if (data_in) begin
15         //transition to next state
16         next_state = STATE_3;
17         four_ones_det = 1'b0;
18     end
19     else begin
20         //retain same state
21         next_state = STATE_2;
22         four_ones_det = 1'b0;
23     end
24 end // case: STATE_2

1  end // case: STATE_2
2  STATE_3 : begin
3      if (data_in) begin
4          //transition to next state
5          next_state = STATE_0;
6          four_ones_det = 1'b1;
7      end
8      else begin
9          //retain same state
10         next_state = STATE_3;
11         four_ones_det = 1'b0;
12     end
13 end // case: STATE_3
14 endcase
15 end // block: next_state_bl
16
17 // Current Register Block
18 always @(posedge clk) begin : current_state_bl
19     if (reset)
20         current_state <= #1 STATE_0;
21     else
22         current_state <= #1 next_state;
23     end
24 endmodule
```

Secuenciamiento y Control

Guía para la codificación de FSM

Ejemplo de Aplicación de la Guía para FSM

- El siguiente ejemplo de código implementa el mismo diseño que el anterior pero utilizando codificación one-hot y procesa la salida en bloque de asignación continua separado. Sólo algunos estados se escriben para la demostración. Para el mapeo efectivo de hardware, se utiliza una instrucción case inversa donde cada caso sólo se evalúa para TRUE o FALSE

```
1 always@(*)
2 begin
3     next_state = 4'b0 ;
4     case (1'b1) // Synopsys parallel_case full_case
5         current_state[S0]:
6             if (in)
7                 next_state[S1] = 1'b1 ;
8             else
9                 next_state[S0] = 1'b1 ;
10            current_state[S1]:
11                if (in)
12                    next_state[S2] = 1'b1 ;
13                else
14                    next_state[S1] = 1'b1 ;
15            endcase
16        end
17        // Separately coded output function
18        assign output = state[S3];
```

Rep. Algorítmica de máquinas de estado



Introducción

- El mapeo de hardware de procesamiento de señales y algoritmos de comunicación es el objetivo de este curso. Estos algoritmos generalmente se mapean en arquitecturas con multiplexado en el tiempo. Estas arquitecturas requieren del diseño de FSMs para generar las señales de control para el datapath.
- En muchos diseños los controladores tienen funcionalidad similar a algoritmos. Esta funcionalidad también implica lógica para decisiones. Los diagramas de burbuja (diagramas de estado) no son lo suficientemente flexibles para describir el comportamiento complejo de estas máquinas de estado.
- Para estos casos se utiliza una notación de máquina de estado algorítmica (ASM). Es un tipo de gráfico del tipo de diagrama de flujo que describe el comportamiento ciclo a ciclo de un algoritmo. Cada paso de transición entre un estado y otro o al mismo estado lleva un ciclo de clock.
- El ASM esta compuesto por 3 bloques básicos: Rectángulos, diamantes y óvalos. Se utilizan flechas para interconectar estos bloques. Cada rectángulo representa un estado y la salida de un estado se escribe dentro de un rectángulo. La salida de un estado siempre es una salida incondicional, que se valida cuando la transición de la máquina de estado se representa por el respectivo rectángulo.

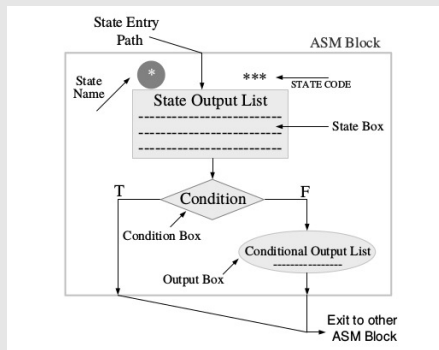
Rep. Algorítmica de máquinas de estado

Representación de ASM

Introducción

- En la figura se muestra una relación entre los componentes básicos de una representación ASM. La condición en el diamante puede terminar en un óvalo que significa una salida condicional.

Representación ASM



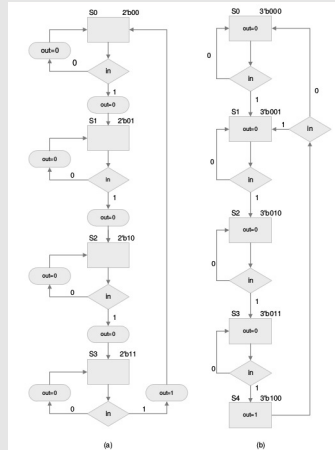
Introducción

- La forma de representación ASM es muy descriptiva y provee un mecanismo para el diseño paso a paso y sistemático de circuitos síncronos. Una vez definida puede ser fácilmente traducida a código RTL Verilog.
- Existen herramientas que pueden convertir una representación ASM en código RTL Verilog.
- Consideremos nuevamente el ejemplo del contador de 4 bits en 1 que hemos estado trabajando. Este ejemplo se podría representar en un formato ASM tanto para Mealy como para Moore. La máquina de Moore requerirá un estado adicional . La figura (a) muestra el ejemplo para Mealy y (b) corresponde a una FSM de Moore.

Rep. Algorítmica de máquinas de estado

Representación de ASM

Ejemplo ASM Moore y Mealy



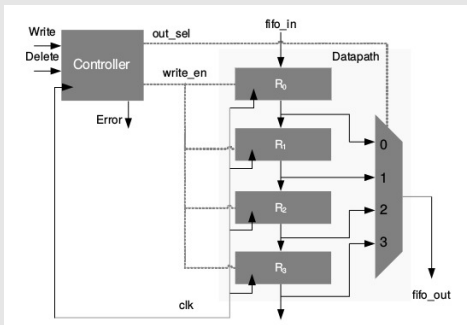
Rep. Algorítmica de máquinas de estado

Ejemplos de ASM

Ejemplo : Diseño de FIFO de 4 Entradas

- La siguiente figura muestra la división entre datapath y unidad de control para una fifo de 4 entradas. Las entradas al controlador son dos operaciones, Write y Del (Escritura y borrado). La escritura mueve el dato en fifo_in a la cola de la FIFO y el borrado elimina el dato de la 'cabeza'.

Ejemplo ASM Moore y Mealy



Ejemplo : Diseño de FIFO de 4 Entradas

- La 'cabeza' de la FIFO siempre esta disponible en `fifo_out`. La escritura en la FIFO llena o la eliminación en la FIFO vacía causa una condición de error. La ejecución de Write y Del al mismo tiempo también causa una condición de error
- El datapath consiste de 4 registros R_0 , R_1 , R_2 y R_3 y un multiplexor para seleccionar la cabeza de la FIFO.
- La entrada a la FIFO se almacena en R_0 cuando se activa `write_en`. Esta señal `write_en` también mueve el resto de las entradas en la fifo una posición hacia abajo.
- Con cada nueva escritura y borrado de los datos, el controlador necesita seleccionar la 'cabeza' de su nueva ubicación utilizando una señal `out_sel` al multiplexor.

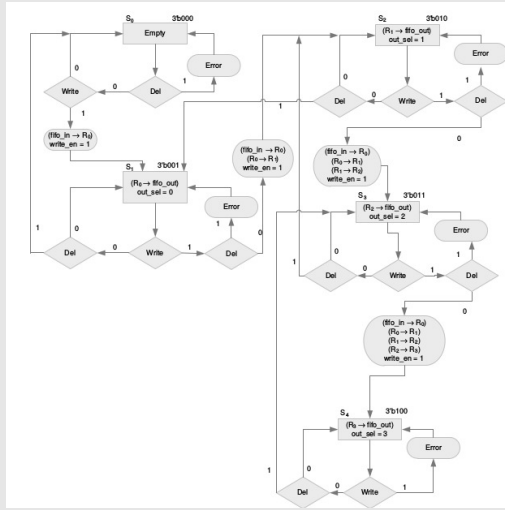
Ejemplo : Diseño de FIFO de 4 Entradas

- El estado inicial del FSM es S0 e identifica el estado en el cual la FIFO se encuentra vacía.
- Cualquier solicitud de Del para una FIFO vacía causa una condición de error como se muestra en el diagrama.
- En el caso de una solicitud de escritura, el controlador activa write_en y el valor de fifo_in pasa al registro R_0 . El controlador selecciona R_0 como salida asignando un valor de 3'b00 a out_sel.
- En la solicitud de escritura, el controlador también produce la transición a S1.
- Cuando la FSM se encuentra en S1, una operación Del lleva a la FSM nuevamente a 0 y un Write provoca la transición a S2, lo mismo para S2 a S3. En S3 una nueva solicitud de escritura produce un error ya que la FIFO está llena.
- Escritura y Borrado al mismo tiempo genera un error.

Rep. Algorítmica de máquinas de estado

Ejemplos de ASM

ASM FIFO de 4 entradas



Rep. Algorítmica de máquinas de estado

Ejemplos de ASM

Código Verilog FIFO de 4 entradas

```
1 // Combinational part only for S0 and default state is given
2 always @(*)
3 begin
4     next_state=0;
5     case(current_state)
6     'S0:
7     begin
8         if(!Del&& Write)
9         begin
10             next_state = 'S1;
11             write_en = 1'b1;
12             Error= 1'b0;
13             out_sel = 0;
14         end
15     else if (Del)
16     begin
17         next_state='S0;
18         write_en =1'b0;
19         Error = 1'b1;
20         out_sel=0;
21     end
22     else
23     begin
24         next_state='S0;
25         write_en=1'b0;
26         out_sel = 1'b0;
27     end
```

```
1 // Similarly , rest of the states are coded //
2 default:
3     begin
4         next_state='S0;
5         write_en = 1'b0;
6         Error = 1'b0;
7         out_sel =0;
8     end
9 endcase
10 end
11 // Sequential part
12 always @(posedge clk or negedge rst_n)
13     if(!rst_n)
14         current_state <= #1 'S0;
15     else
16         current_state <= #1 next_state;
```

Optimización de FSM



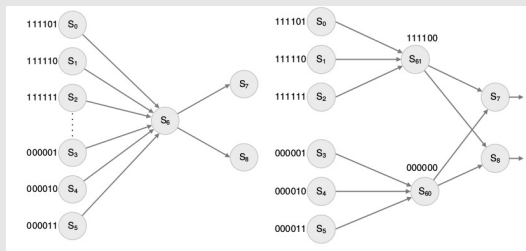
Optimización de FSM

Introducción

- Existen varias formas de implementar una FSM. El diseñador, basado en una serie de objetivos de diseño puede seleccionar entre las opciones y luego la herramienta de síntesis realiza una optimización sobre la implementación de la FSM.
- La herramienta, basada en criterios de optimización, transforma la FSM a una versión con la misma funcionalidad pero topológicamente distinta. Generalmente se optimiza para baja potencia y área reducida. La disipación dinámica de potencia en la lógica digital depende de la actividad de switching. Las herramientas, mientras generan una minimización de los estados para reducir el área, también apuntan a minimizar la actividad de switching total.
- De la variedad de algoritmos de optimización disponibles, uno es encontrar el estado crítico que está conectado a muchos estados. En esta situación la transición a este estado crítico puede causar grandes distancias hamming, generando gran actividad de switching. El algoritmo, luego de identificar este estado crítico lo separa en dos para reducir la distancia hamming y luego analiza la FSM para evaluar si esta separación produce un gran incremento de área.

Optimización de FSM

Optimización de FSM por parte de la Síntesis



En la parte (a) se muestra la FSM original y (b) muestra la FSM transformada con reducción de actividad de switching, como resultado de las transiciones de estado, formada por una división del estado crítico.



The image shows a detailed 3D rendering of an Intel Agilex FPGA chip. The chip is a square package with various components labeled on its surface. At the top, it says 'Intel® Agilex™ FPGA'. Below this, there's a '10nm FPGA Fabric' block. To the left of the fabric is a 'CPU Cores' block. To the right is a 'PCIe Gen 5' block. Below the fabric is a 'DDR' block. Above the fabric is an 'HBM' block. To the right of the fabric is an 'Other Chiplet' block. There are also several 'EMIB' (Embedded Multi-Die Interconnect Bridge) blocks connecting different parts of the chip. The background is a dark blue network of lines and nodes, suggesting a digital or interconnected environment.

Diseño para Verificación

Introducción

- Desde el punto de vista de la verificación, se recomienda agregar a una FSM la posibilidad de reset y la capacidad de reportar el estado. Para proveer a una FSM con capacidad de reset, debería existir una entrada que provoque que la FSM realice una transición desde cualquier estado a un estado definido inicial.
- Una máquina de estado con capacidad de reportar el estado también aumenta la posibilidad de verificación de un diseño. Una FSM con esta característica siempre retorna el estado actual para una entrada de consulta definida.
- A la metodología general para verificación de diseños digitales, se le puede sumar un documento de especificación sobre la máquina de estado que defina el comportamiento de la FSM para todas las secuencias posibles de entradas. Esto requiere al desarrollador generar una serie de secuencias de operación que puedan verificar todas las transiciones de una FSM.

Introducción

- Para una verificación inicial a nivel módulo white-box, el estímulo debería llevar un registro de todas las transiciones correspondientes a estados internos de la FSM y las entradas y salidas previas a la aparición de una falla. Esto ayuda al diseñador a encontrar de manera más fácil un bug.
- Para una secuencia de entradas, utilizando las especificaciones de diseño, el diseñador debe encontrar la correspondiente secuencia de salidas para escribir los casos de prueba.
- Varias medidas de Coverage (cobertura de código) se utilizan para evaluar la eficiencia de las pruebas. Se puede disponer de herramientas de coverage que el diseñador puede incorporar al flujo de RTL. Estas herramientas ayudan a identificar áreas no verificadas de una implementación de FSM. El diseñador entonces puede generar nuevos casos para verificar esas áreas.

Diseño para Verificación

Métricas de Coverage para validación de diseños

Coverage de Código

Es la métrica más simple que valida que todas las líneas del código se ejecutan al menos una vez. Es común de verificar a través de simuladores. Si bien una cobertura de código de 100% es una buena medida inicial, no es un indicador suficiente de la calidad de la verificación.

Coverage de Toggle

Esta medida verifica si cada bit de todos los registros produjo una transición de 0 a 1 y de 1 a 0 al menos una vez. Cubrir esta es bastante difícil ya que el contenido de los registros suele ser el resultado de funciones complejas. Suelen utilizarse pruebas random para verificar esta métrica.

Coverage de Estados

Esta etapa de cobertura lleva el registro de la frecuencia de las transiciones a los diferentes estados. El objetivo es asegurarse que las verificaciones provoquen las transiciones de la FSM por todos los estados al menos una vez. En muchas FSM se puede llegar a un mismo estado desde diferentes estados, por lo tanto este tipo de coverage no garantiza la verificación completa de la FSM

Diseño para Verificación

Métricas de Coverage para validación de diseños

Coverage de Transición o de Arco

Esta métrica requiere que la FSM utilice todas las transiciones del diseño al menos una vez.

Coverage de Path

Esta es la medición de cobertura más intensiva para evaluar un diseño. Comenzando desde el estado inicial, atravesando todos los path del diseño lleva a una verificación muy completa. En muchos diseños con un número grande de estados, la generación exhaustiva de vectores de prueba para atravesar todos los posibles paths puede no ser posible, por lo que se requieren técnicas para generar caminos no triviales para verificar.

Una de las técnicas es 'reducción del espacio de estado'. Esto reduce la FSM a una FSM más pequeña a través de unir distintos nodos en uno. Entonces se verifica el coverage de path para esa FSM reducida.

Métodos para Reducción de Potencia



Métodos para Reducción de Potencia

Potencia de Switching

- La distribución de la red de clock y el switching de clock son las dos fuentes de mayor consumo de potencia. La potencia de switching está dada por la siguiente expresión:

$$P_{avg} = n\alpha_{avg} \frac{f}{2} C_{avg} V_{dd}^2$$

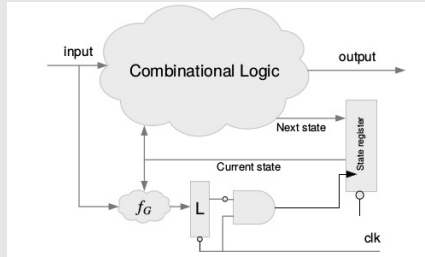
- En la expresión n es número total de nodos en el circuito, α_{avg} es la actividad promedio de switching en un nodo, f es la frecuencia de clock y V_{dd} es el voltaje de la fuente. Es de suponer, que eliminando el clock en la red de distribución y reduciendo la actividad de switching promedio se puede reducir la disipación de potencia promedio.
- En un diseño con máquinas de estado complejas, la máquina de estado es la parte que mayor potencia utiliza. Este consumo se puede reducir implementando técnicas de clock gating.

Métodos para Reducción de Potencia

Clock Gating

Clock Gating

- Los diseños basados en FSM en general esperan a un evento externo para cambiar de estado. En estos diseños, cuando ese evento no sucede, se verifica constantemente la entrada se realizan transiciones sobre un mismo estado en cada ciclo de clock. Esto disipa potencia de manera innecesaria.
- Analizando las especificaciones de la FSM, el diseñador puede apagar la FSM cuando se encuentra en un lazo continuo de transición en el mismo estado. Esto requiere detener el clock al aplicar una compuerta con la salida de una lógica de detección de lazo sobre un mismo estado (self-loop detection logic).



Métodos para Reducción de Potencia

Clock Gating

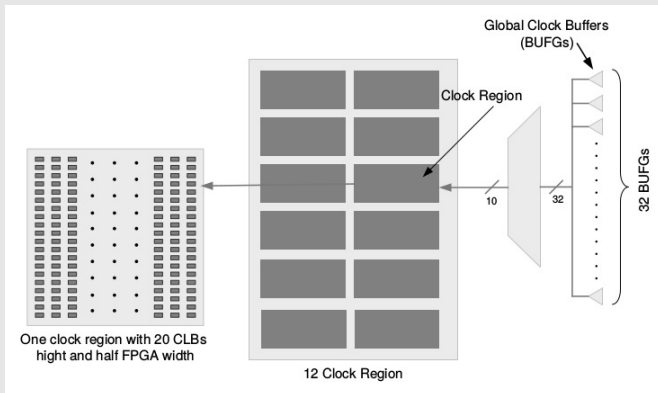
Clock Gating

- En la figura anterior, el registro L en el diseño se utiliza para bloqueo de glitches en la señal de salida debida a la lógica de gating f_G . La función f_G implementa la lógica para determinar si la máquina se encuentra en un lazo recursivo sobre su mismo estado. Esta lógica puede ser diseñada para actuar en un caso con mayor probabilidad de quedar en un self-loop. Esto requiere un conocimiento de la estadística de transiciones de la FSM implementada.
- En el caso de las FPGA se han desarrollado técnicas de clock gating muy efectivas. Las FPGA tienen redes de clocks dedicadas para llevar el clock a todos los bloques con skew y jitter bajos. Por ejemplo en Xilinx Virtex 5 se tienen bloques embebidos como RAM, DSP48 y IOs, pero para configurar los bloques básicos existen los CLSBs (Configurable Logic Blocks).
- Cada CLB consiste de dos partes, cada parte contiene cuatro LUTs de 6 entradas y cuatro flip-flops. La FPGA tiene 32 buffers de clock globales (BUFGCEs) para soportar esta cantidad de clocks generados internamente o externamente. La FPGA se divide en múltiples regiones de clocks.

Métodos para Reducción de Potencia

Clock Gating

Clock Gating en FPGA

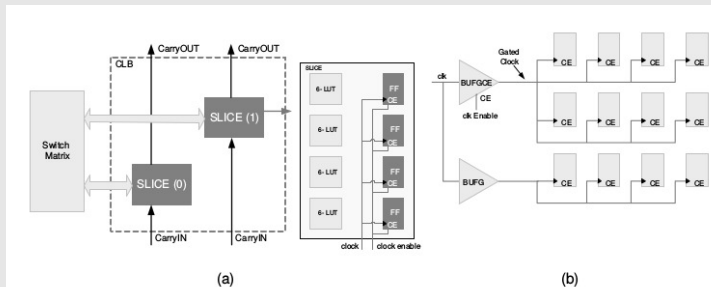


Métodos para Reducción de Potencia

Clock Gating

Clock Gating en FPGA

- Todos los FF en una porción tienen una señal de clock enable (a). Aunque deshabilitar esta señal puede ahorrar algo de potencia, la potencia dinámica disipada en el clock va a continuar.
- Para ahorrar potencia cada buffer de clock tiene una señal de enable. Esta señal evita que el clock se conecte a la red de distribución.



Métodos para Reducción de Potencia

Descomposición de FSM

Descomposición de FSM

- Esta técnica propone particionar la FSM en múltiples sub-FSMs de manera que la máquina de estados permanezca apagada en la mayoría de las sub-FSM por un período de tiempo considerable.
- En el mejor caso esto lleva a que solo es necesario clocking de una de las sub-FSM para un ciclo de clock determinado. Deshabilitar los árboles de clocks para el resto de las sub-FSM por medio de clock gating puede reducir significativamente el consumo.
- El ejemplo ilustra el agrupamiento de una FSM en sub-FSM utilizando un método gráfico de particionamiento. La técnica utiliza la probabilidad de transiciones de estados para unir un grupo de nodos que se encuentran relacionados entre sí hacia un sub-FSM individual. Considere una FSM dividida en M particiones de sub-FSM cada una con estados $S_0, S_1, \dots, S_{M-1}$. Para codificar de manera sencilla, el número de bits asignados para cada registro de estado de cada sub-FSM es:

$$\max \{ \log_2 S_0, \log_2 S_1, \dots, \log_2 S_{M-1} \}$$

Métodos para Reducción de Potencia

Descomposición de FSM

Descomposición de FSM

- En algunos casos las FSM son jerárquicas en su algoritmo, lo cual permite particionar de manera óptima la FSM para reducir potencia. En este caso el algoritmo se particiona en niveles, entonces al efectuarse transiciones desde un nivel más alto a uno más bajo, el resto de las sub-FSM no serán utilizadas en esa trama o buffer de datos.

Ejemplo partición de FSM jerárquica

